

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Parametric imitation learning via behavior cloning can suffer from poor generalization to out-of-distribution states due to compounding errors during deployment. We show that reusing the training data during inference via a semi-parametric retrieval-based imitation learning approach directly addresses this challenge. We present **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric retrieval-based imitation learning approach that reparameterizes the imitation learning problem in terms of local neighborhood structure rather than direct state-to-action mappings. Instead of learning a global policy, DARP trains a model to predict actions based on k -nearest neighbors from expert demonstrations, their corresponding actions, and the relative distance vectors between neighbor states and query states. Crucially, DARP delivers these gains without relaxing any of the assumptions made for standard behavior cloning: it requires no additional data collection, no online expert feedback, and no task-specific knowledge. Across a broad evaluation spanning continuous control (MuJoCo), robotic manipulation (Robosuite, RoboCasa), high-dimensional visual features, and a real-world assembly task, DARP improves over standard behavior cloning in *every* setting we test, with head-to-head gains of 15–46%. The improvement is not only consistent but widens as tasks become harder: on the real-world square-table task DARP more than doubles the success rate of behavior cloning (92% vs. 44%), and on high-dimensional visual inputs its average margin ($\sim 35\%$) exceeds that on low-dimensional states ($\sim 22\%$). Code and demos will be released upon acceptance.

1 INTRODUCTION

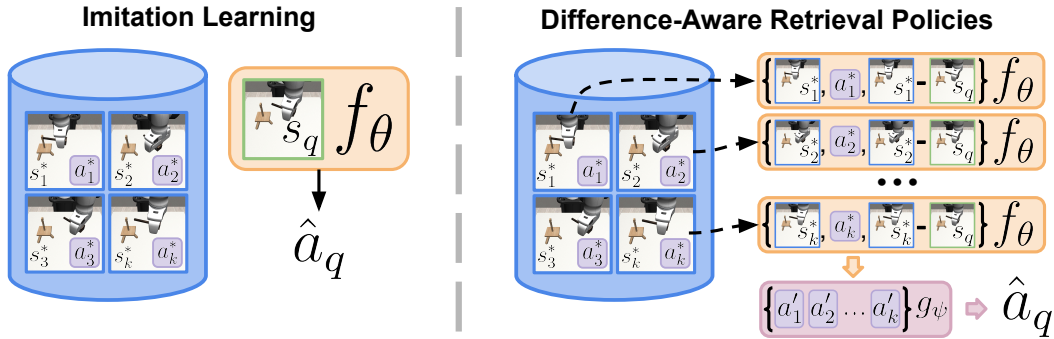


Figure 1: **Overview of DARP:** Unlike standard BC (left), which discards the expert dataset after training and maps each query state directly to an action, DARP (right) retains the dataset and reuses it at inference via a retrieval-based reparameterization centered on difference vectors between query states and retrieved neighbors. Around each query point s_q , DARP retrieves a local neighborhood of expert state-action pairs and predicts from the differences to those neighbors—the design choice that our experiments identify as the primary source of its 15–46% gains over BC.

Imitation learning via behavior cloning (BC) (Pomerleau, 1991) has enabled robots to learn complex, dexterous behaviors from expert demonstrations (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). Yet despite its simplicity, BC often proves brittle in practice, especially for long-horizon tasks (Ross et al., 2011). The core issue is *covariate shift*: small errors accumulate during rollouts, driving the agent into states not well represented in the demonstration data (Spencer et al., 2021; Ross et al., 2011). In such out-of-distribution regions, BC policies are highly unstable, producing unreliable and high-variance behavior that frequently leads to failure. This failure mode is exactly what our method targets, and, as we show empirically, closing it translates into large and consistent success-rate gains across every domain we study.

This problem is well recognized, and many approaches have been proposed to mitigate compounding error (Ross et al., 2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020). However, these typically go beyond the standard BC assumptions, requiring simulators, interactive experts, large quantities of sub-optimal data, or strong task-specific structure. By contrast, our goal is to remain in the pure BC regime: learn only from expert state-action pairs, with no additional supervision or feedback. The central question is thus: *can we reduce the variance of BC policies using only the original demonstration dataset?*

From a statistical standpoint, BC minimizes only the supervised risk on expert states. This controls bias on the training distribution, but leaves variance unchecked: in low-density regions of the state space (which are often encountered during closed-loop rollouts), the learned policy can oscillate arbitrarily. A natural remedy is to enforce *smoothness*, so that nearby states yield similar predicted actions (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). This discourages spurious fluctuations and improves rollout stability. Several approaches to encourage smoothness have been explored, see related work in Section 4.

Although sometimes effective, each has drawbacks: augmentation does not guarantee consistency, global priors can blur distinct behaviors, temporal penalties only act along time (not space), and explicit graph regularizers require tuning extra smoothness hyperparameters.

A complementary line of work contrasts global and local learning. “Global” supervised models (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) attempt to compress the entire demonstration dataset into a single parametric function, which is typically brittle under distribution shift. “Local” methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) instead adapt predictions to the structure of the dataset itself, consulting neighborhoods of similar states and generating outputs from non-parametric or semi-parametric operations on the training distribution of expert behavior. This locality offers robustness since it avoids reliance on a single parametric function, but it also has limitations: its effectiveness inherently depends on the distance metric, naive averaging of neighborhood can blur distinct actions and struggle to represent multimodality, and treating neighbors only in terms of their absolute states can limit generalization. Our experiments make this trade-off concrete: the purely non-parametric baselines (R&P and LWR) trail standard BC on the MuJoCo suite, confirming that locality alone is not enough—the gains we report come specifically from coupling retrieval with a learned, difference-aware predictor.

We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, which combines the robustness of local methods with the stability of regularized global policy learning. At inference time, rather than predicting actions to execute only from the current query state via a feedforward pass on a parametric function, DARP (Fig. 1) first retrieves a set of k neighbors from the training corpus and then predicts k actions conditioned on tuples of (neighbor state, associated action, and difference from the query state). These neighbor-informed predictions are then aggregated in a permutation-invariant manner to produce a single robust action prediction. This design both grounds predictions in observed data (due to non-parametric retrieval) and implicitly enforces local consistency (due to parametric action prediction, conditional on the retrieved neighbors). We show that doing so reduces variance without requiring any additional assumptions beyond those made for standard Behavior Cloning – we need no additional data, online supervision, or task-specific knowledge. In spectral terms, this form of neighbor aggregation approximates a Laplacian filter on the k -NN (k -nearest neighbor) graph of expert states, providing a parameter-free form of smoothing that adapts to the local density and geometry of the dataset.

We provide both theoretical and empirical evidence that, while operating under the same requirements as behavior cloning, DARP improves performance considerably by reducing variance and enhancing

robustness to distribution shift. Our analysis formalizes the connection to Laplacian regularization, showing that DARP implicitly applies a fixed low-pass spectral filter that suppresses high-frequency variance. Empirically, the pattern is uniform and sizable: measured head-to-head against standard behavior cloning under identical data and evaluation protocols, DARP wins on *every* task and domain we test, with gains of 15–46% spanning continuous control (MuJoCo), robotic manipulation (Robosuite, RoboCasa), and high-dimensional visual imitation (Robosuite with image state). The advantage is not confined to a favorable regime—it holds across low- and high-dimensional state representations, across MLP, transformer, and Gaussian-mixture policy classes, and it in fact *grows* on the harder visual tasks (an average margin of $\sim 35\%$ versus $\sim 22\%$ on low-dimensional states). We further show that DARP is a general, scalable architecture that naturally extends to image-based domains with rich policy classes such as transformers and Gaussian mixture models, and we report a careful set of ablations that isolate which components—most notably the difference vectors and permutation-invariant aggregation—drive these gains, providing general-purpose insights into retrieval-based algorithms for sequential decision-making problems.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

In this work, we instantiate a new class of imitation learning methods that combine the strengths of “global” parametric learning and “local” retrieval-based learning. We propose a new architecture and a simple training objective that operate under exactly the same requirements as typical behavior cloning, yet deliver both provable guarantees and large empirical gains—15–46% over BC across every domain we test, as detailed in Section 3. In this section we derive the theoretical guarantees from first principles. Readers primarily interested in the empirical evidence may skip to Section 2.5 for a self-contained summary of DARP before proceeding to the quantitative results in Section 3.

As a warmup, we define the problem setting (Section 2.1) and discuss a variant of regularized imitation learning (Section 2.2) that imposes additional structure from the data for improvements in variance, generalization, and stability. In Section 2.3, we then show how the benefits of explicitly regularized learning can be implicitly accomplished by modifying policy *architecture* rather than the objective. Finally, in Section 2.4 we introduce our practical algorithm DARP, which realizes these benefits through a semi-parametric retrieval augmented architecture that can be generally applied to imitation learning with modern neural networks and generative modeling tools.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We operate in the typical imitation learning setting, formalized by a finite-horizon Markov Decision Process (MDP), $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and P_0 is the initial state distribution. A *policy* maps a state to a distribution of actions $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$ so as to maximize task-relevant objectives (for brevity, we drop the subscript θ when discussing the general function class f). We assume access to expert human-provided demonstrations $\mathcal{D}^*$ as a collection of state-action pairs: $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$. We use the notation s^* and a^* specifically to denote states and actions belonging to the expert dataset. The behavior cloning (Pomerleau, 1991) algorithm learns a policy f_θ from this dataset by casting imitation as a typical supervised learning problem — $\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. While the distribution class of f_θ can be an arbitrary complex generative model (Lipman et al., 2023; Chi et al., 2024), we will start with a Gaussian parameterization for the sake of simplicity¹.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

While behavior cloning minimizes only the supervised imitation loss over expert states drawn from $\mathcal{D}^*$, such an objective alone does not control how the policy behaves on states that deviate from the manifold of expert states. In practice, accumulating errors lead the agent to out-of-distribution regions where a BC policy may act arbitrarily, especially for overparameterized neural networks (Ross & Bagnell, 2010).

¹We show that this can be relaxed in Section 2.4

To mitigate this, we note that behavior cloning enforces function evaluations only at the training states, but it does not explicitly take into account the relationship between states (and their corresponding actions) in a neighborhood, thereby ignoring the underlying data manifold. To incorporate this information into policy learning, let us consider a modified objective that introduces a regularization term that explicitly encourages *local consistency* of predictions: nearby states in the expert dataset should be mapped to similar actions. This intuition leads to the following neighborhood-regularized loss ($\mathcal{L}_{\text{MRIL}}$), where the standard imitation learning objective ($\mathcal{L}_{\text{BC}}$) is combined with an additional smoothness penalty ($\mathcal{L}_S$) enforcing predictions to respect the geometry of the dataset rather than relying solely on pointwise supervision.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk}(\mathcal{L}_{\text{BC}})} + \lambda \underbrace{\mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer}(\mathcal{L}_S)}, \quad (1)$$

where $\ell(f(s^*), a^*)$ is the supervised imitation loss, $\mathcal{N}_k(s^*)$ are the k -nearest neighbors of s^* from the expert dataset, and the weights $w_i(s^*)$ are normalized kernel weights based on the state differences — $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$. As we discuss briefly below (and in detail in Appendix A.1.1), this corresponds to a form of *manifold regularization* or Laplacian smoothing, where the policy is penalized for high-frequency variation across the neighborhood of expert states. This manifold regularization provably leads to improvements in policy variance, stability, and generalization.

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^2$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof sketch. We defer the detailed proof to Appendix A.1.1, but provide a brief sketch. The key idea of the proof is to first show that the smoothness regularizer directly corresponds to a graph Laplacian penalty on a graph constructed by a k -nearest neighbor (k -NN) affinity matrix defined by the kernel

² L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

w_i . Next, we show that as the number of samples tends to infinity, this graph Laplacian penalty converges to the weighted Dirichlet energy (Belkin & Niyogi, 2008; Zhou et al., 2003). Minimizing this Dirichlet energy (1) ensures that the learned f is locally Lipschitz almost everywhere, ensuring smoothness and, in turn, policy stability, and (2) corresponds to Tikhonov regularization, thereby reducing estimator variance, while keeping the bias controlled. $\square$

Intuitively, the smoothness regularizer is not merely penalizing pairwise disagreements between neighbors, but is driving the learned policy to be smooth with respect to the underlying data manifold. In particular, it shrinks the local Lipschitz constant of f along directions where the data density $p(s)$ is high, ensuring that small changes in state lead to small, consistent changes in the predicted action. As a result, the policy generalizes more reliably on in-distribution (ID) states and extrapolates in a structured manner on new out-of-distribution (OOD) states in the neighborhood.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

While our MRIL objective does amortize local learning to provide improvements over vanilla BC, there are two notable drawbacks. First, it requires a hyperparameter λ that must be tuned to balance supervised accuracy and smoothness. Second, the requirement to optimize a modified, regularized objective rather than a standard BC objective may modify the optimization landscape in adverse ways. This raises a natural question: *can we obtain the same benefits conferred by MRIL (Eq. 1), by modifying the policy architecture rather than modifying the objective?*

In this section, we introduce a retrieval-based change in policy architecture that leads to an *implicit* manifold regularization effect (iMRIL), despite using a standard imitation objective. With iMRIL, we can obtain the benefits of Laplacian smoothing (from MRIL) by training on a standard BC objective (as shown in Fig. 2), without introducing λ as an additional hyperparameter for training. We then build on this algorithm to develop a practical instantiation of this method (DARP) in Section 2.4.

iMRIL architecture: The high-level idea behind iMRIL is simple – we propose moving the neighborhood aggregation (averaging) operation from the objective (as in Eq. 1) to the architecture itself. So instead of learning a standard feedforward predictor $f(s)$ that is trained against a neighborhood regularized smoothness objective (Eq. 1), we propose embedding the structure of neighborhood aggregation directly into the parameterization of the action predictor $\hat{f}$ itself, while maintaining the objective as standard imitation learning. iMRIL learns the parameters of a per-state predictor f_θ such that an action predictor explicitly parameterized via neighborhood-aggregation $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$ across nearest neighbor states from the training set $\{s_i^*\}_{i \in \mathcal{N}_k(s^*)}$ generates accurate predictions of the corresponding expert action a^* . With this parameterization, iMRIL optimizes at training time:

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment time, inference can be performed on a new state s_q simply by retrieving the k -NN of s_q from the training set and performing neighborhood aggregation $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*)$. As we show in Section 2.4, the particular parameterization of f is of crucial importance and plays a significant role in the empirical performance of iMRIL— leading to the development of DARP.

Intuitively, we are parameterizing the action predictor $\hat{f}$ as an aggregation of predictions at neighbor states from the training data $f(s_i^*)$, and then learning f . Supervising the post-aggregation function implicitly prevents any f predictions from being arbitrarily non-smooth, conferring the benefits noted in Section 2.2. We prove a direct equivalence of iMRIL to the Laplacian regularization in Section 2.2.

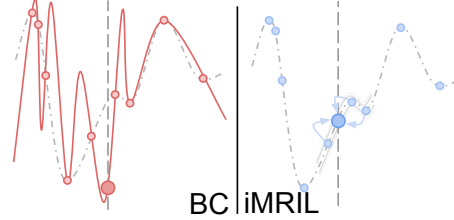


Figure 2: **iMRIL implicitly achieves Laplacian smoothing, reducing variance and enforcing local consistency**, whereas standard BC, lacking any smoothness constraint, admits arbitrarily jagged interpolants. This variance-reducing effect is the mechanism behind the empirical gains reported in Section 3.

Equivalence between iMRIL and MRIL: While we defer a full proof of formal equivalence between MRIL and iMRIL to the Appendix Section A.1, we state our main result and a proof sketch to this effect here.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. We defer full details to Appendix Section A.1.2. The explicit regularizer admits a spectral solution by diagonalizing the k -NN Laplacian, yielding a filter of the form $(1 + \lambda \mu)^{-1}$ on each eigenmode. The implicit objective can be expressed as neighbor aggregation $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which has the same eigenvectors and applies the fixed filter $1 - \mu$. Intuitively, both act as low-pass filters on the graph: modes with small eigenvalues (smooth variation across the data manifold) are largely preserved, while modes with large eigenvalues (rapid, high-variance fluctuations between neighbors) are strongly damped. Thus iMRIL implicitly performs Laplacian smoothing, reducing variance and enforcing local consistency without needing to tune λ . $\square$

Note that the implicit Laplacian smoothing view does not replace the need to learn a policy; rather, it constrains the class of functions that can be represented after aggregation. The neighbor-conditioned network f_θ learns how expert actions vary under local perturbations, proposing locally adapted actions for each neighbor. The aggregation operator then enforces variance reduction by smoothing these proposals across the neighborhood. In this way, learning provides accuracy by correcting local bias, while aggregation provides stability by controlling variance.

2.4 DIFFERENCE-AWARE RETRIEVAL POLICIES: A PRACTICAL INSTANTIATION OF iMRIL FOR IMITATION LEARNING

Given the conceptual framework of iMRIL, we instantiate a practical algorithm for large-scale imitation learning. We build on the objective outlined in Eq. 2 and instantiate a careful choice of (1) parameterization, (2) neighbor aggregation that leads to strong empirical performance.

2.4.1 DIFFERENCE-BASED PARAMETERIZATION OF f_θ

The objective described in Eq. 2 leaves the parameterization and input representations of f_θ open to broad interpretation. We make the observation that the neighborhood aggregation should learn how expert actions vary under local perturbations. This suggests that f_θ should use knowledge of *differences* between a query state and a neighbor state to adaptively propose locally adapted actions for each neighbor. In **Difference-Aware Retrieval Policies (DARP)**, instead of simply parameterizing f_θ by $f_\theta(s_i^*)$, we provide additional context about the optimal neighbor action a_i^* , as well as the *difference* between the query state and the neighbor state $\Delta s_i = s_i^* - s_q$; a predictor f_θ predicts an action candidate a_i' for a query state s_q and a neighbor (s_i^*, a_i^*) using the difference information as $a_i' = f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)$.

Let $\mathcal{N}_k(s_q)$ be the index set of the k -nearest neighbors retrieved according to some distance function $d(s_q, s_i^*)$.³ For generating predictions with DARP, we can then perform neighborhood aggregation (as outlined in Section 2.3) to predict an action for any query state s_q

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

At training time, this can be used to define a straightforward imitation learning objective from the expert dataset $\mathcal{D}^*$:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

where we optimize the parameters of the predictor f_{θ} , minimizing the discrepancy between the predicted action $\hat{a}_q$ and the optimal action a_q^* . Given the simplicity of the objective, any parameterization can be used for f_{θ} ; in our case, standard feedforward or convolutional neural networks. This difference-based parameterization is not a minor detail: our ablations (Section 3) show it is one of the two largest contributors to DARP’s gains, with performance dropping sharply when the difference vectors are removed. At inference time, we generate actions by retrieving the k -NN and applying the neighborhood aggregation operation defined in Eq. 3.

2.4.2 GOING BEYOND LINEAR AGGREGATION

While the process of neighborhood aggregation thus far has been restricted to averaging over neighborhood predictions $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$, this is a special case of a broader class of permutation-invariant aggregation functions $g_{\psi}(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$. For instance, g_{ψ} could be parameterized with more expressive set-compliant neural models like the set transformer (Lee et al., 2019) or DeepSets (Zaheer et al., 2017). This suggests a generalization of the prediction model in Eq. 3 as $\hat{a}_q = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. Besides benefits in expressivity, generalizing from a simple averaging operation to a parametric aggregation model g_{ψ} allows for the representation of richer action distributions (e.g Gaussian mixture models (Pignat & Calinon, 2019) or diffusion models (Chi et al., 2024)) than the Gaussian distribution that is implicit to the L_2 -regression objective defined in Eq. 5. Rather than predicting $\hat{a}_q$ directly, DARP can predict the parameters α of an action distribution $p(a_q; \alpha)$ – for instance the means, covariances, and weights for a Gaussian mixture model, or the score function for a diffusion model. This allows DARP to perform maximum likelihood training of multimodal action distributions rather than just unimodal L_2 -regression:

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_{\theta}(s_q^*))], \text{ where } \alpha_{\theta}(s_q^*) = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)}) \quad (6)$$

Inference for a query state s_q can be performed by sampling $\hat{a}_q \sim p(\cdot; \alpha_{\theta}(s_q))$, constructing $\alpha_{\theta}(s_q) = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$ from a set of neighbors retrieved at test time. We refer readers to Appendix Section A.3 for detailed training pseudocode.

2.5 SUMMARY

We show that encouraging local consistency when fitting a behavior cloning policy corresponds to manifold regularization (Laplacian smoothing) and provably improves variance, smoothness, and stability over standard BC (Theorem 1)—the same three quantities whose empirical benefit is confirmed by the consistent 15–46% success-rate gains in Section 3. Rather than adding an explicit regularization term (and its attendant hyperparameter λ) to the loss, we show that the identical smoothing effect can be induced implicitly by building neighborhood aggregation into the policy architecture itself (Theorem 2). Thus, we propose the following algorithm: at training

³In our work we use the Euclidean distance in a pre-trained embedding space, although other neighborhood functions are also applicable. We refer the reader to Appendix Section A.2.1 for a thorough discussion of design decisions in constructing neighborhood sets via retrieval.

time, for each expert state-action pair (s_q^*, a_q^*) , DARP retrieves the k -nearest neighbor states from $\mathcal{D}^*$, computes difference vectors $\Delta s_i = s_i^* - s_q^*$ from each neighbor to the query state, passes each neighbor tuple $(s_i^*, a_i^*, \Delta s_i)$ through a network f_θ to produce candidate actions, and finally aggregates these candidates via a permutation-invariant function g_ψ to predict the final action. This is trained only with the standard imitation learning objective.

3 EXPERIMENTAL EVALUATION

Next, we evaluate DARP in order to answer three key questions: **Q1:** Does DARP consistently outperform standard behavior cloning, and by how much?, **Q2:** Do these gains hold as we move to more complex state representations and action distributions?, **Q3:** Which architectural components are responsible for the observed performance gains? To answer these, we compare DARP head-to-head against standard BC and a suite of retrieval and smoothness baselines under matched data and evaluation budgets, across multiple domains using low-dimensional state representations, high-dimensional image features, and diverse action representations. Our evaluation spans continuous control tasks (MuJoCo), robotic manipulation (Robosuite, RoboCasa), and specially designed discontinuous environments that stress-test the neighbor-based approach. As a preview, DARP is the top-performing method in every comparison that follows.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) tasks entail controlling various legged figures in multiple embodiments to achieve forward locomotion on a flat plane. These tasks include: Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) tasks all entail a single robotic arm manipulating objects. In the Stack task, the goal is to put a smaller cube on top of a larger one. In the Threading task, the goal is to manipulate a thin, needle-like tool and insert it into a small ring. In the Square Peg task, the goal is to manipulate a square wooden block with a hole in the center and place it onto a square peg.

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) tasks all entail a single robotic arm manipulating objects in a randomized kitchen setting. In the Drawer task, the goal is to close an open drawer. In the Door task, the goal is to close an open microwave oven door. In the Stove task, the goal is to twist a knob to turn off a stove burner.

Baseline Comparisons: To situate DARP precisely, we compare against baselines that span the full spectrum from purely non-parametric retrieval to fully parametric policies, so that any gain can be attributed to a specific design choice rather than to retrieval in general: (1) **R&P (Sridhar et al., 2025)**, which directly executes the action of the single nearest neighbor (non-parametric); (2) **LWR (Pari et al., 2022)**, which performs locally weighted regression over retrieved neighbors (non-parametric); (3) **BC**, standard parametric behavior cloning—our primary point of comparison; (4) **REGENT (Sridhar et al., 2025)**, a transformer-based in-context learning method conditioned on retrieved neighbors; and (5) **MRIL**, the explicitly smoothed variant of DARP from Section 2.2. This set lets us separate the contribution of retrieval alone (R&P, LWR), of parametric learning alone (BC), and of the difference-aware, learned-retrieval combination that defines DARP.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

In this experiment, we test DARP’s core hypothesis on tasks with low-dimensional state representations, where the distance metrics between states are well-defined and interpretable. This evaluation spans locomotion tasks from MuJoCo and robotic manipulation tasks from Robosuite and RoboCasa with data generated with MimicGen (Mandlekar et al., 2023). To make the comparison as conservative as possible, the aggregation function g here is only a simple unweighted average of the neighbor action predictions a' —the gains reported below are therefore attributable to the difference-aware reparameterization itself, before any expressive aggregator is introduced.

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **DARP attains the best score on every MuJoCo environment, and DARP Set Transformer is the runner-up on three of four.** Rewards for DARP versus BC and all baselines on MuJoCo continuous-control tasks with low-dimensional state (best per column in bold). The margins are large and one-directional: DARP improves on BC from 1063.23 to 5515.41 on HalfCheetah (a $5.2\times$ increase), from 2376.20 to 4383.28 on Ant, from 2658.40 to 4894.01 on Walker, and from 2313.65 to 3545.57 on Hopper. The purely non-parametric baselines (R&P, LWR) trail BC on most tasks, and MRIL—while consistently above BC—is in turn surpassed by DARP everywhere, isolating the difference-aware architecture as the source of the top scores. Scores are averaged across 100 independent trials with 95% confidence intervals. Parametric policies use Multi-Layer Perceptrons; for diffusion-policy results, see Section 8.

The comparison is decisive: DARP is the single best method on every environment we test, and the gap over standard behavior cloning is consistent in direction and large in magnitude. On robotic manipulation (Table 2) the improvement is 15–25 success-rate points per task, while on locomotion (Table 1) the reward gains are even more pronounced—most strikingly on HalfCheetah, where DARP reaches 5515.41 against BC’s 1063.23. Two contrasts sharpen the picture. First, the purely non-parametric baselines (R&P and LWR) not only fail to match DARP but generally fall below BC itself, confirming that retrieval without a learned difference-aware predictor is insufficient. Second, MRIL—which adds explicit smoothing—almost always edges out vanilla BC, yet DARP surpasses MRIL on every task, indicating that the implicit, architecture-level reparameterization captures the benefit of smoothing and then adds a further margin on top.

Having established these gains on compact states, we next ask whether they persist under the practical instantiation of Section 2.4 as we scale to higher-dimensional input representations such as images.

486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539



Figure 3: The “DrawerClose” RoboCasa task, in which the arm must close an open drawer; DARP reaches 85% success here versus 54% for BC (Table 2).



Figure 4: The “Threading” Robosuite task, in which the arm must insert a needle into a small hole; DARP raises success from BC’s 37% to 63% on low-dimensional state (Table 2).

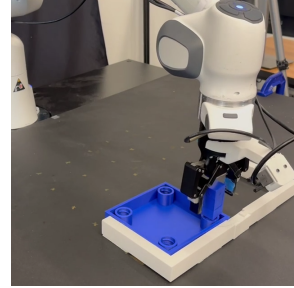


Figure 5: Real-world FurnitureBench square-table assembly task: the robot picks up a table leg and screws it into a corner hole of the tabletop. This is our hardest setting and shows the widest margin—DARP succeeds 92% of the time versus 44% for BC (Table 2).

Method	RoboCasa			Robosuite			Real
	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: **DARP beats BC on all seven manipulation tasks spanning three environments, with the margin largest in the real world.** Head-to-head success percentages using low-dimensional state features (RoboCasa & Robosuite: 100 trials; Real: 50 trials; best per column in bold). Every column favors DARP: RoboCasa Drawer 85 vs. 54, Door 45 vs. 29, Stove 43 vs. 28; Robosuite Stack 72 vs. 47, Threading 63 vs. 37, Peg 62 vs. 46; and, most notably, the real-world Square Table task at 92 vs. 44—more than double the success rate of BC. RoboCasa and Robosuite policies use Multi-Layer Perceptrons; the real-world result uses a diffusion policy (see Section 8).

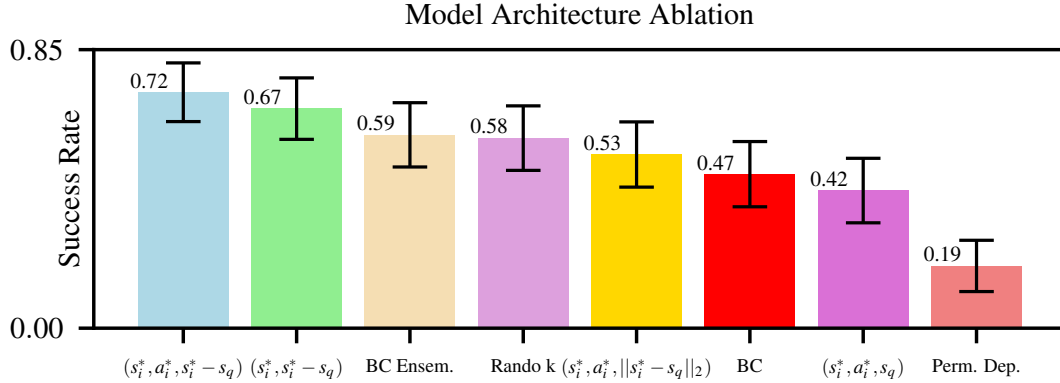


Figure 6: **Difference vectors and permutation-invariant aggregation are the two components that account for most of DARP’s gain.** Removing each design choice in turn shows that the largest drops follow from discarding the difference vectors $(s_i^*, a_i^*, s_i^* - s_q)$ or replacing the permutation-invariant aggregator, whereas removing neighbor actions costs comparatively little; random-neighbor selection collapses performance, confirming that meaningful retrieval is essential. Success rate is averaged across 100 trials on the Robosuite Stack environment with 95% confidence intervals.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)

High-Dimensional Visual Input Representations. To test whether DARP’s advantage survives beyond compact, low-dimensional states, we evaluate on simulated robotic manipulation tasks using R3M image embeddings (Nair et al., 2022), where each state is a high-dimensional feature vector extracted from visual observations (Table 3). The result is the opposite of the degradation one might fear for a neighbor-based method in high dimensions: DARP not only continues to beat BC on all three tasks, its average margin here ($\sim 35\%$) is *larger* than the corresponding low-dimensional Robosuite margin ($\sim 22\%$). Concretely, DARP more than triples BC’s peg-insertion success (52% vs. 17%) and exactly doubles its threading success (76% vs. 38%). In other words, the harder and higher-dimensional the perception problem, the more DARP pulls ahead of standard BC.

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: Success rates (%) on vision-based Robosuite tasks (R3M features, 100 trials). DARP improves on BC on all three tasks (Stack 75 vs. 44, Threading 76 vs. 38, Peg 52 vs. 17), more than tripling the peg-insertion rate.

Multi-modal Action Distributions. The advantage also carries over to tasks that demand genuinely multimodal action distributions: on the Push-T environment, DARP outperforms behavior cloning by more than 20 points (details in Appendix A.2.2), showing that the gains are not tied to unimodal, Gaussian-style action modeling.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: To pinpoint *which* design choices produce DARP’s gains rather than merely confirming that they exist, we run a controlled ablation that turns off one component at a time: (1) full DARP; (2) DARP without the neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with random neighbors instead of a distance metric; (5) DARP using the L_2 norm of the distance vector; (6) the BC baseline, which sees only the query state s_q ; (7) DARP conditioned on the query state alone rather than the query-to-neighbor difference vector; and (8) DARP with a permutation-*dependent* (i.e., **not** permutation-invariant) aggregator. Holding everything else fixed, each variant reveals the marginal value of the component it removes; Figure 6 reports the resulting success rates on a common task.

The ablation resolves the question of *where* the gains come from. Two components dominate: removing the difference vectors or swapping the permutation-invariant aggregator for a permutation-dependent one each produces the sharpest drops in success rate, identifying them as the primary

drivers of DARP’s advantage. Neighbor actions contribute a smaller but positive increment. The importance of meaningful retrieval is underscored by the random-neighbor variant, which performs poorly—so the gains depend on selecting the *right* neighbors, not merely on conditioning on some neighbors. Together these results attribute DARP’s improvement over BC to a specific, reproducible combination of design choices rather than to incidental factors.

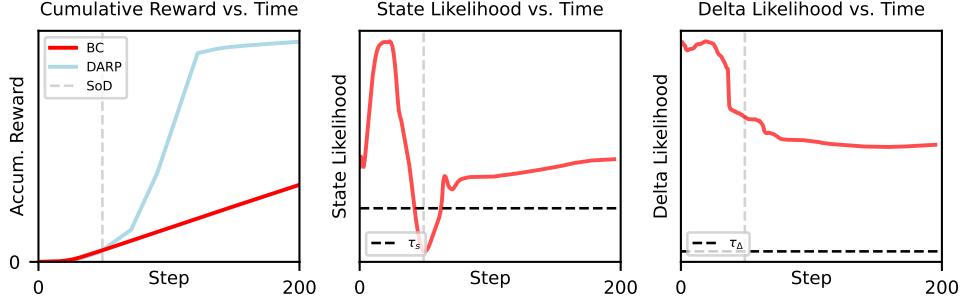


Figure 7: On the Robosuite Stack task, BC and DARP follow identical reward curves until a single divergence point, after which BC fails and DARP succeeds. The vertical dashed line marks the step of divergence (“SoD”). Precisely at the SoD the query state is out-of-distribution in absolute terms (state likelihood $< \tau_s$) yet in-distribution in difference terms (delta likelihood $> \tau_\Delta$)—the exact regime the difference-aware reparameterization is designed to exploit, and the mechanism behind the success-rate gaps reported above.

Divergence Analysis: To explain *why* DARP outperforms BC, we examine the exact rollout steps where BC fails but DARP succeeds. We define the “step of divergence” (SoD) as the point at which the two methods’ rewards begin to differ significantly. We set τ_s and τ_Δ to the 1st percentile of training-set likelihoods (i.e., only 1% of deltas seen at training are less likely than τ_Δ). The pattern is perfectly consistent: in all six rollouts across two tasks (Robosuite Stack and MuJoCo Hopper), the SoD query state is out-of-distribution in absolute state likelihood ($< \tau_s$) yet in-distribution in delta likelihood ($> \tau_\Delta$). This 6-of-6 result gives a concrete mechanistic account for the aggregate success-rate gains: BC fails precisely where the difference-based representation remains well-supported, which is exactly where DARP keeps predicting accurately (see Figure 7 for reward drift, SoDs, and likelihoods on one task). Appendix A.2.4 reports further robustness experiments that corroborate this account.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: Non-parametric IL algorithms demonstrate surprising performance by leveraging local structure. VINN Pari et al. (2022) explores locally weighted regression for imitation, showing surprising results in image embedding spaces, and MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and achieve rapid generalization. SEABO Lyu et al. (2024) uses retrieval methods to perform offline RL by rewarding transitions close to neighbors to form a reward function. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) perform non-parametric retrieval and finetuning from large unlabeled datasets, enabling generalization through test-time training. DARP differs from the above in its unique parameterization of retrieved states into $(s_i^*, a_i^*, s_i^* - s_q)$ tuples and in learning a semi-parametric policy rather than relying purely on non-parametric aggregation or test-time training. This distinction is not merely conceptual: in our experiments the purely non-parametric baselines (R&P, LWR) fall below standard BC on the MuJoCo suite, whereas DARP attains the best score on every task—empirically separating the variance reduction of local methods from the generalization of parametric policies and showing that DARP captures both.

Smoothness-Constrained Policy Learning: Much recent literature has explored explicit smoothness constraints to improve policy stability and robustness. L2C2 Kobayashi (2022) considers model-free RL under local Lipschitz continuity constraints, achieving smoothness and noise robustness without sacrificing expressiveness, while Asadi et al. (2018) proposed a similar methodology for model-based RL models with Lipschitz constraints. CCIL Ke et al. (2024a) extends these ideas to generate synthetic corrective labels for imitation learning using a Lipschitz-constrained dynamics model. This has also been scaled up to humanoid controllers Chen et al. (2024) to reduce shakiness on deployment.

DARP differs from these methods by enforcing smoothness implicitly through an architecture change while using standard imitation learning objectives. In a direct comparison against CCIL, the strongest smoothness-based baseline (see Appendix), DARP matches or exceeds its relative improvement over BC on all four MuJoCo tasks—by a wide margin on HalfCheetah (418.7% vs. 5.4%) and Ant (84.5% vs. 12.6%)—indicating that the implicit, hyperparameter-free approach is at least as effective as explicit smoothness constraints.

In-Context Learning Methods: Recent work has explored non-parametric retrieval from the perspective of in-context imitation learning. REGENT Sridhar et al. (2025) investigates retrieval-augmented generalization by incorporating retrieved states, actions, and rewards into a causal transformer, while DPT Lee et al. (2023) uses supervised pretraining for transformers to predict actions given query states and in-context datasets, effectively learning how to explore. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylius & Johns (2025), and KAT Di Palo & Johns (2024). These methods aim to quickly adapt to new tasks and environments, whereas DARP focuses on accomplishing higher performance and stability on standard imitation learning.

5 CONCLUSION

We introduced **D**ifference-**A**ware **R**etrieval **P**olicies for Imitation Learning (**DARP**) (DARP), a nearest-neighbor-based algorithm that reparameterizes imitation learning in terms of relative differences between query states and their nearest neighbors rather than learning direct state-to-action mappings, and we prove that it implicitly achieves Laplacian smoothing. Empirically, the evidence is uniform: across continuous control, simulated manipulation, high-dimensional visual inputs, and a real-world assembly task, DARP improves over standard behavior cloning in every setting we test, with head-to-head gains of 15–46%. Three findings stand out. First, DARP consistently beats BC on low-dimensional state, and is the top-performing method in every such comparison against non-parametric and smoothness baselines alike. Second, the advantage is robust to—and even amplified by—greater complexity: it holds across state representations, multimodal action distributions, and policy classes, with its average visual-input margin ($\sim 35\%$) exceeding its low-dimensional margin ($\sim 22\%$), and it more than doubles BC’s success on the real-world task (92% vs. 44%). Third, controlled ablations attribute these gains specifically to the difference vectors and permutation-invariant aggregation, identifying a concrete and reproducible source of the improvement.

6 REPRODUCIBILITY STATEMENT

To make every reported number independently verifiable, we will release upon acceptance the supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating expert demonstrations for Robosuite tasks via MimicGen, and modality transformation code. On NVIDIA L40 and L40s GPUs, the released code reproduces the paper’s results exactly, with the sole exception of the transformer-based results (REGENT, Set Transformer), which are non-deterministic and may differ slightly from the reported numbers.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.

- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.
- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.
- Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkrit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.

756 Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple
757 search-based method for offline imitation learning. In *International Conference on Learning*
758 *Representations*, 2024.

759 Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteyio Akinola, Yashraj Narang, Linxi Fan, Yuke
760 Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human
761 demonstrations. In *7th Annual Conference on Robot Learning*, 2023.

762 Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous
763 control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under
764 review as a conference paper at ICLR 2018.

765 Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot
766 sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference*
767 *on Learning Representations*, 2025.

768 Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal
769 visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski
770 (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*,
771 volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL
772 <https://proceedings.mlr.press/v205/nair23a.html>.

773 Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi,
774 Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist
775 robots. In *Robotics: Science and Systems (RSS)*, 2024.

776 Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The
777 surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and*
778 *Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.
779 XVIII-052.

780 Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imita-
781 tion. *IEEE Robotics and Automation Letters*, 2019. URL [https://arxiv.org/pdf/1904.](https://arxiv.org/pdf/1904.10716.pdf)
782 [10716.pdf](https://arxiv.org/pdf/1904.10716.pdf). Preprint version.

783 Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural*
784 *Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL [https://doi.org/10.](https://doi.org/10.1162/neco.1991.3.1.88)
785 [1162/neco.1991.3.1.88](https://doi.org/10.1162/neco.1991.3.1.88).

786 Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the*
787 *thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR
788 Workshop and Conference Proceedings, 2010.

789 Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and
790 structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson,
791 and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial*
792 *Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15
793 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL [http://proceedings.mlr.](http://proceedings.mlr.press/v15/ross11a/ross11a.pdf)
794 [press/v15/ross11a/ross11a.pdf](http://proceedings.mlr.press/v15/ross11a/ross11a.pdf).

795 Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to
796 dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data:*
797 *Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328.
798 Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL [https://link.springer.](https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33)
799 [com/chapter/10.1007/978-1-4612-2660-4_33](https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33).

800 Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bag-
801 nell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872,
802 2021. URL <https://arxiv.org/abs/2102.02872>.

803 Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-
804 augmented generalist agent that can act in-context in new environments. In *The Thirteenth*
805 *International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.

-
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

To prove Theorem 1, we first start with a well-known result (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \longrightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^4$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof. We prove each claim in turn.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top Lf,$$

where $L = D - W$ is the graph Laplacian, and W is the k -NN affinity matrix. By Lemma 1, this equals the empirical Dirichlet energy of f on the k -NN graph. It is well known (Zhou et al., 2003; Belkin & Niyogi, 2008) that such a quadratic penalty is equivalent to Tikhonov regularization with respect to the graph Laplacian norm $\|f\|_L^2 = f^\top Lf$. In statistical learning theory, adding a Tikhonov penalty strictly reduces the variance of the estimator compared to the unregularized solution while keeping the bias term controlled. Thus MRIL enjoys smaller estimator variance than vanilla BC, which uses no such penalty.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This is the weighted Dirichlet energy of f on $\mathcal{M}$. If this integral is finite, f belongs to the Sobolev space $H^1(\mathcal{M}, p^2)$, and in particular f is locally Lipschitz almost everywhere with

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore minimizers of $\mathcal{L}_{\text{MRIL}}$ have uniformly bounded local Lipschitz constants along high-density regions of the state space. By contrast, minimizers of vanilla BC have no such constraint: any oscillatory interpolant that matches the training data exactly yields the same supervised risk, so arbitrarily large Lipschitz constants are possible.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is only minimized on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

For MRIL, the Laplacian penalty enforces

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

⁴ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(h^2) + O(\Delta_t)).$$

Since the constant multiplying Δ_t is strictly smaller under the smoothness constraint, the cumulative error grows strictly slower than in vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas it is linear for vanilla BC.

Conclusion. Claims (i)–(iii) establish that MRIL yields lower variance, uniform smoothness control, and sublinear rollout error accumulation compared to vanilla BC, completing the proof. $\square$

Kernel choice. For the IC smoothness regularizer, we adopt a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This choice is standard in manifold regularization (Belkin & Niyogi, 2008; Zhou et al., 2003) and ensures that the graph Laplacian penalty converges to the Dirichlet energy in the continuum limit. In practice, we found this default to be stable across tasks, though other kernels (e.g., uniform k -NN or exponential decay) yield qualitatively similar results.

A.1.2 PROOF OF THEOREM 2

We begin with the required Lemmas establishing the spectral form of explicit Laplacian regularization and neighbor aggregation.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus λ directly determines the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U \Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = U c$, $a^* = U b$ in this basis. The objective becomes

$$\|U c - U b\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back gives the stated expansion. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = S f$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2} A D^{-1/2}$ and $S = D^{-1} A = I - L_{\text{rw}}$ where $L_{\text{rw}} = D^{-1} L$ is the random-walk Laplacian. Since L_{rw} and L share the same spectrum up to similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Thus for each mode u_j , $S u_j = (1 - \mu_j) u_j$, yielding the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus in the graph Fourier basis, neighbor aggregation corresponds to multiplying each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

On the other hand, Lemma 2 shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda\mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The key difference is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To see the equivalence, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. Moreover, for large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes even more strongly by driving them toward zero, providing a sharper low-pass effect than explicit regularization.

Therefore, DARP's aggregation step is mathematically equivalent to implicit Laplacian regularization with fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

DARP can therefore be viewed as a form of *locally adaptive implicit regularization*: rather than introducing an explicit global weight λ , its neighbor aggregation step enforces smoothness automatically through the graph structure. The effective regularization strength varies with local degree and neighborhood geometry, adapting to the density of the expert demonstrations. Spectrally, this corresponds to the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$, which suppresses high-frequency modes more aggressively than any fixed explicit λ . Figure 8 illustrates this comparison, showing how DARP achieves sharper low-pass filtering without the need for hyperparameter tuning.

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

The selection of the distance function $d(s_q, s_i^*)$ to select k neighbors is crucial. While we find that simple Euclidean distance between states can work, in our experiments, we use a slightly modified algorithm that takes advantage of the fact that we are working with sequences of states and incorporates history in our distance calculation.

Suppose we have a query trajectory $S_q = (\dots, s_{q-1}, s_{q,0})$ where $s_{q,0}$ is the current query state s_q . Now suppose we want to calculate $d(s_q, s_i^*)$, where s_i^* is some state from the expert dataset. We first find the trajectory this state is from—call this S_j^* —and the index of s_i^* in this trajectory—call this t . Thus,

s_i^* can be rewritten as $s_{j,t}^*$. Given some lookback parameter ℓ which denotes how many past states we want to consider, we get:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This is simply the accumulation of Euclidean distances of the current and last $\ell - 1$ states from both the query trajectory and the source trajectory, assuming valid indices. Of course, in practice, we generally want to put more emphasis on more recent states, as we want them to be more influential in the selection of neighbors. Thus, given some rate of exponential decay $\gamma \geq 0$, we have

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

See Figure 9 for an experimental analysis on how the success rate in an environment changes as these parameters are swept.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

We test DARP’s ability to handle complex action distributions on the Push-T task (Chi et al., 2024), which requires representing multi-modal action distributions (Figure 10). Here DARP uses a Set Transformer head that predicts the parameters of a Gaussian Mixture Model. The result confirms that the advantage extends to explicitly multimodal modeling: DARP scores 70 against BC’s 48 (Table 4), a 22-point absolute improvement (Q2), showing that difference-aware retrieval composes cleanly with rich, multimodal action heads rather than being tied to unimodal regression.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A key concern for neighbor-based approaches is performance in environments with strong discontinuities, where states that are close in Euclidean distance may require drastically different actions—the setting most adversarial to retrieval. We stress-test exactly this case with a modified version of D4RL’s Umaze environment (Figure 11).

Even in this deliberately worst-case discontinuous environment, DARP reaches a 57% success rate against BC’s 25% (Q3, Table 5)—more than double BC’s success. That the advantage survives, rather than collapsing, under geometry designed to mislead a neighbor-based method indicates that the difference vectors and permutation-invariant aggregation let DARP tell appropriate neighbors from inappropriate ones even when spatial proximity does not imply action similarity.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

To directly measure DARP’s robustness to accumulated error, we roll out a BC agent in an environment where it is known to fail and, every k steps, fork the environment to launch a DARP agent from BC’s current (increasingly off-distribution) state. Figure 12 shows that DARP recovers and scores highly from nearly every fork point, even deep into BC’s drift away from the expert support. The takeaway is quantitative and mechanistic: DARP does not merely start better, it can rescue trajectories that BC has already begun to lose, which is precisely the behavior needed to convert BC’s compounding-error failures into the success-rate gains reported in the main results.

A.2.5 COMPARISON WITH CCIL

Table 6 compares DARP against CCIL, a baseline that explicitly induces smoothness, using CCIL’s own reported percent improvement over BC. On a like-for-like basis, DARP equals CCIL on Walker and beats it on the remaining three environments, with the gap widening as the task gets harder for CCIL—most dramatically on HalfCheetah, where DARP delivers a 418.7% improvement over BC against CCIL’s 5.4% (a nearly 78× larger relative gain). This shows that DARP’s implicit, hyperparameter-free smoothing is at least as effective as, and often far stronger than, explicit smoothness constraints.

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **Relative improvement over BC: DARP vs. CCIL.** Head-to-head on standard MuJoCo benchmarks, using the improvement-over-BC scores reported by CCIL. DARP matches CCIL on Walker (84.1% each) and exceeds it substantially on the other three tasks—Hopper (53.2% vs. 32.6%), Ant (84.5% vs. 12.6%), and HalfCheetah (418.7% vs. 5.4%)—so DARP is never worse than this explicit-smoothness baseline and is far better on three of four tasks.

A.2.6 DISTANCE METRIC SENSITIVITY

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **Distance Metric Sensitivity.** Success rates on the Robosuite Stack task with R3M features. Both distance choices clear BC (47%) by a wide margin—Euclidean at 75% (+28) and cosine at 70% (+23)—and differ from each other by only 5 points, so DARP’s advantage does not hinge on the particular metric used for retrieval.

While all experiments in this paper use Euclidean distance to select neighbors, it is natural to ask whether the gains depend on that choice, particularly in high-dimensional embeddings such as R3M where cosine similarity is common. Table 7 shows they do not: switching to cosine similarity costs only 5 points (70% vs. 75%), and both variants remain far above BC’s 47%. DARP’s advantage is therefore a property of the difference-aware architecture, not an artifact of a finely tuned distance metric.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP composes with diffusion policy, and the gains stack.** Success rates over 50 trials on the state-based Robosuite Stack, Threading, and Peg Insertion tasks. DARP improves over its backbone in both regimes: with an MLP backbone it beats BC (62/38/40 vs. 34/12/28), and with a diffusion backbone it beats plain diffusion (82/72/66 vs. 52/44/54), attaining the best result in every column. Because this ablation uses far fewer demonstrations than Table 2, the absolute BC and DARP numbers are not directly comparable across the two tables; the relevant quantity here is the within-table gap over each backbone.

While the main experiments use an MLP backbone, diffusion policy (Chi et al., 2024) is a state-of-the-art model class for imitation learning, especially for manipulation. Table 8 shows DARP is

complementary to—not competing with—diffusion: pairing the two yields the strongest model in every column. Swapping DARP’s MLP backbone for a diffusion backbone adds 20, 34, and 26 success-rate points on Stack, Threading, and Peg Insertion respectively, and the resulting DARP-with-diffusion policy beats standard BC by 48, 60, and 38 points on those same tasks. The improvement from difference-aware retrieval and the improvement from a more expressive backbone are thus additive rather than redundant.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **DARP (MLP) is far cheaper than diffusion policy at matched task.** Runtime costs measured on identical hardware. DARP trains in 0.559 s/epoch versus diffusion’s 5.610 ($\approx 10\times$ faster) and runs inference in 0.00437 s/step versus 0.15700 ($\approx 36\times$ faster), so the accuracy gains of the MLP variant come at an order-of-magnitude lower compute cost.

Beyond accuracy, DARP with an MLP backbone is dramatically cheaper than diffusion—about $10\times$ faster to train and $36\times$ faster at inference (Table 9)—so practitioners can obtain DARP’s gains without diffusion’s runtime overhead.

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **Results with human demonstrations.** Success rates on the Robosuite Stack task when training on human-collected data rather than RL-collected data. DARP maintains its edge over BC (60% vs. 45%, a 15-point gain), showing the advantage is not an artifact of RL-generated experts.

The expert demonstrations for the MuJoCo and Robosuite experiments are collected by an optimal Reinforcement Learning policy, which raises the question of whether DARP’s advantage depends on that clean, RL-generated data. It does not: trained instead on human demonstrations for the Robosuite Stack task, DARP still outperforms standard BC by 15 points (60% vs. 45%, Table 10), confirming that the gain persists under the noisier, more realistic demonstrations practitioners typically have.

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **With hyperparameters chosen by validation loss, DARP still beats BC.** Mean scores on Hopper and Stack. Selecting training epochs and retrieval settings purely by validation loss—i.e., without oracle tuning on test performance—DARP leads BC on both tasks (Hopper 805.69 vs. 507.49, a $\sim 59\%$ gain; Stack 0.31 vs. 0.12, more than $2.5\times$), so the reported gains do not rely on favorable hyperparameter selection.

Figure 9 shows that a few extreme retrieval settings (e.g., very few neighbors) can push DARP below BC, so it is fair to ask whether our reported gains depend on hand-picked hyperparameters. Table 11 answers this: selecting settings by validation loss alone recovers strongly performant configurations, with DARP beating BC on both Hopper and Stack. The advantage is thus reproducible under a principled, test-blind selection rule rather than requiring oracle tuning.

A.3 PSEUDOCODE

We provide pseudocode of the DARP algorithm, see Algorithm 1.

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \text{argmin-}k; d(s_q^*, s_j^*)$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **DARP’s overhead over BC is small and scales sub-linearly in k .** Training (s/epoch) and test (s/step) times for BC and for DARP at varying k on Hopper and Stack. On Stack, increasing k from 100 to 2000—a $20\times$ larger neighborhood—raises training time only from 0.556 to 0.758 s/epoch and leaves inference roughly flat (~ 0.0045 s/step); relative to BC, inference costs about 0.003 s/step more, a negligible price for the accuracy gains reported in the main results.

Table 12 quantifies DARP’s computational overhead at both training and inference. The cost of retrieval grows sub-linearly in k , and inference stays firmly in the real-time regime: on Robosuite Stack at $k = 500$, a single step takes about 0.00437 s on our hardware—over 230 Hz, well above typical robot control rates. In practical terms, DARP delivers its accuracy gains while remaining fast enough for real-time deployment and only marginally slower than BC.

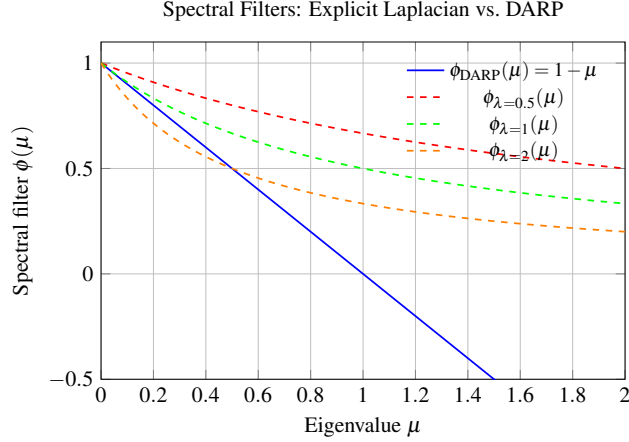


Figure 8: DARP applies a sharper low-pass spectral filter than explicit Laplacian regularization: its fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$ suppresses high-frequency modes more strongly than $(1 + \lambda\mu)^{-1}$ for any fixed λ , yet requires no hyperparameter tuning.

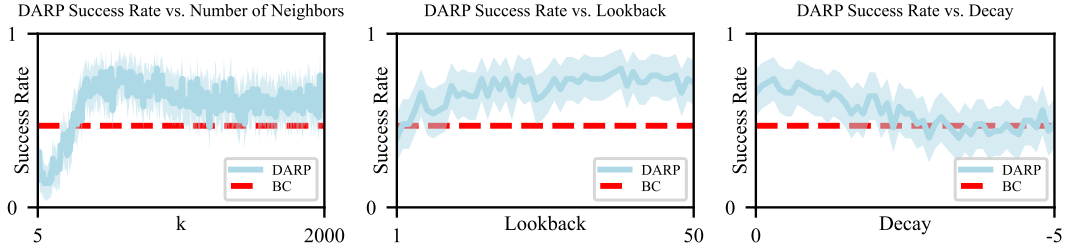


Figure 9: How DARP’s success rate responds to each retrieval hyperparameter (Robosuite Stack, 50 trials, 95% confidence intervals). (Left) Neighbor count k : performance is low with few neighbors, peaks around $k \approx 500$, and plateaus just above BC as k approaches the full dataset size—so DARP meets or exceeds BC across a broad range of k and is best at an interior optimum. (Center) History length: adding lookback steadily helps, and DARP only drops below BC when almost no history is used. (Right) Decay rate on older observations: very aggressive decay effectively removes history and pushes performance down toward the no-lookback (sub-BC) regime, whereas moderate decay is best. Across all three sweeps, the settings that fall below BC are narrow extremes, while the bulk of the range favors DARP.

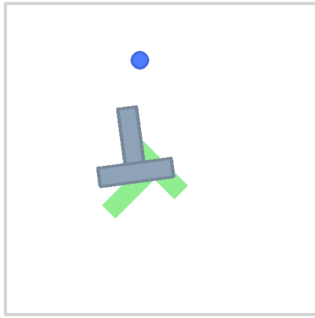


Figure 10: **Push-T Environment.** The goal is to control the blue circle to push the T-shaped block.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T Results.** Over 100 trials, DARP scores 70 vs. BC’s 48—a 22-point (relative $\sim 46\%$) gain on this multimodal task, with an identical ± 8 interval for both methods.

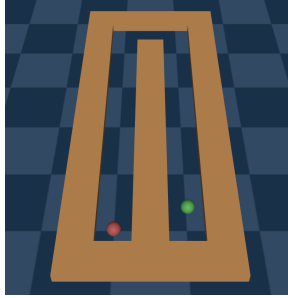


Figure 11: **Long maze environment.** The goal is to move a force-actuated ball from the green start to the red destination.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long maze results.** Over 100 trials, DARP reaches 57% success versus BC’s 25%—more than double the success rate—even in this discontinuous, retrieval-adversarial environment.

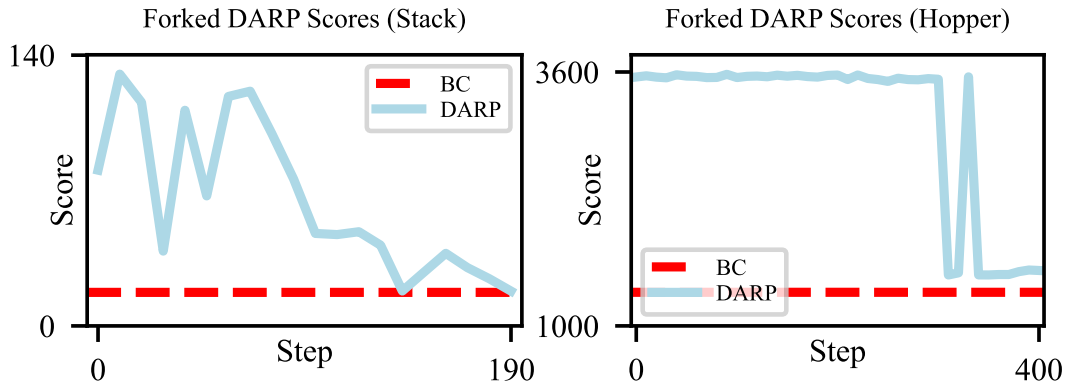


Figure 12: On two tasks (Robosuite Stack and MuJoCo Hopper), we roll out a failing BC agent and fork the environment every $k = 10$ steps to launch DARP from BC’s drifting state. DARP recovers to a high score from almost anywhere along BC’s trajectory—remaining able to succeed until roughly the midpoint of the Stack rollout and about 80% of the way through the Hopper rollout—quantifying how far into BC’s failure DARP can still rescue the episode.